

Resumen primeras clases + toma de notas

SCM – Software Configuration Management

- SCM = Software Configuration Management → Gestión de Configuración de Software.
- La idea principal es mantener la integridad del producto de software durante todo su ciclo de vida.
- El software va cambiando constantemente → cada cambio puede generar nuevas versiones.
- Necesitamos poder saber qué cambió, cuándo cambió y qué versión tenemos.
- También sirve para mantener la trazabilidad de los cambios.

¿Qué administra SCM?

- Ítems de configuración → código fuente, documentos, datos, etc.
- Versiones.
- Configuraciones.
- Cambios.
- Ramas (branches).
- Líneas base.
- Releases.

Línea base

- Es una configuración que fue revisada y aprobada y que sirve como referencia para seguir desarrollando.
- Es como tener una "foto" del estado estable del software.
- Se identifica mediante un tag.
- Sirve también para hacer rollback y poder volver a una versión estable anterior.

Branch / Rama

- Permite bifurcar el desarrollo.
- Se puede usar para desarrollar una funcionalidad, corregir un error o experimentar sin modificar directamente la versión principal.
- Después se pueden integrar las ramas mediante un merge.
- Buena práctica: mantener la rama principal como versión estable.

SCM en Agile

- En Agile cambia un poco la forma de pensar SCM.

- No se busca controlar a los desarrolladores, sino ayudar al equipo a hacer seguimiento y coordinar el desarrollo.
- Como los equipos ágiles son auto-organizados, se busca que SCM acompañe al equipo.

En Agile SCM:

- Responde a los cambios en lugar de intentar evitarlos.
- Se busca automatizar todo lo posible.
- Se busca entregar software de manera frecuente y rápida.
- Elimina desperdicio → no hacer tareas que no agreguen valor.
- Da trazabilidad.
- Da feedback sobre calidad, estabilidad e integridad.
- Es responsabilidad de todo el equipo, no de una sola persona.

Continuous Integration

- Los desarrolladores integran código frecuentemente en un repositorio compartido.
- Cada integración debería ser verificada mediante pruebas automatizadas.
- La idea es detectar errores temprano.
- El software debería estar siempre en un estado entregable.

Continuous Delivery

- El software queda preparado para poder pasar a producción en cualquier momento.
- Hay automatización de pruebas.
- La diferencia con Continuous Deployment es que en Delivery todavía existe una decisión humana para hacer el despliegue.

Continuous Deployment

- El despliegue a producción se hace automáticamente.
- Se utilizan pipelines con una serie de pasos.
- Busca que los cambios lleguen rápido y frecuentemente al usuario.

Requerimientos en ambientes ágiles

- En Agile los requerimientos no se definen todos desde el principio.
- Se van descubriendo y refinando a medida que avanza el proyecto.
- La idea es definir "solo lo suficiente" para poder comenzar.
- Lo importante es entregar valor de negocio, no simplemente muchas funcionalidades.

Diferencia con lo tradicional

Tradicional:

- Se intenta definir todo al principio.
- Requerimientos detallados y cerrados.
- Se busca cumplir el plan.
- Mucha documentación.
- El cambio suele verse como un problema.

Ágil:

- Los requerimientos son evolutivos.
- Se descubren progresivamente.
- Se prioriza entregar valor.
- Se acepta que los requerimientos pueden cambiar.
- Se trabaja mucho con el cliente/usuario.
- Se busca feedback constantemente.

¿Por qué no definir todo al principio?

- Porque muchas veces se desarrollan funcionalidades que después el usuario nunca utiliza.
- Eso genera desperdicio de tiempo y esfuerzo.
- Por eso hay que priorizar lo que realmente genera valor.

Product Backlog

- Es una lista priorizada y ordenada de requerimientos.
- Arriba están los requerimientos de mayor prioridad.
- Los que están abajo tienen menor prioridad y pueden cambiar con el tiempo.
- A medida que avanzamos pueden:
 - aparecer nuevos requerimientos,
 - eliminarse otros,
 - cambiar de prioridad.

Product Owner

- Es una figura muy importante en la gestión ágil.
- Se encarga de entender las necesidades del negocio.
- Prioriza los requerimientos según el valor que aportan.
- Decide qué conviene hacer primero.

Just in Time

- "Analizo cuando lo necesito, ni antes ni después".
- No tiene sentido detallar completamente un requerimiento que todavía no sabemos si va a cambiar.
- Cuando un requerimiento adquiere suficiente prioridad, recién ahí lo analizamos y detallamos más.
- Sirve para reducir desperdicio.

User Stories

- Son una forma de expresar requerimientos desde el punto de vista del usuario.
- Se escriben en lenguaje natural.
- Estructura básica:

Como [rol] quiero [objetivo] para [beneficio].

- Lo importante no es solamente lo que está escrito en la historia, sino también la conversación que existe alrededor de ella.

Criterios de aceptación

- Definen los límites de una User Story.
- Indican qué debe cumplirse para considerar que la funcionalidad satisface lo que necesita el usuario.
- No deberían indicar cómo implementar la solución.
- Ayudan al desarrollador a saber cuándo parar de agregar funcionalidad.
- También ayudan a los testers a definir las pruebas.

INVEST

Una User Story debería ser:

- I – Independent: independiente de otras historias.
- N – Negotiable: se negocia el qué, no el cómo.
- V – Valuable: debe aportar valor al cliente.
- E – Estimable: debemos poder estimar cuánto esfuerzo requiere.
- S – Small: debe ser lo suficientemente pequeña para entrar en un Sprint.
- T – Testable: debe poder comprobarse mediante pruebas.

Definition of Ready

- Una User Story está "Ready" cuando cumple los criterios definidos por el equipo y ya está preparada para ser implementada en un Sprint.

Épicas y Temas

- Épica: User Story demasiado grande que todavía necesita ser detallada/dividida.
- Tema: conjunto de User Stories relacionadas y todavía más general que una épica.

Spikes

- Se utilizan para reducir riesgo o incertidumbre.
- Por ejemplo, investigar una tecnología que el equipo no conoce antes de comprometerse con una funcionalidad.
- Pueden ser técnicas o funcionales.

Estimaciones ágiles

- En Agile las estimaciones se hacen principalmente pensando en el tamaño/complexidad de una User Story.
- No hay que confundir:
 - Tamaño: qué tan grande/complexa es la tarea.
 - Esfuerzo: trabajo necesario.
 - Tiempo: cuánto tarda en calendario.
- Las estimaciones basadas directamente en horas pueden ser más difíciles porque dependen de experiencia, conocimientos, personas, etc.

Story Points

- Se utilizan para representar el tamaño/complejidad relativa de una User Story.
- No representan directamente horas.
- Sirven para comparar una historia con otra.

Poker Planning / Planning Poker

- Es una técnica de estimación grupal.
- Participan principalmente los desarrolladores, porque son quienes conocen el trabajo técnico necesario.
- Cada integrante selecciona una carta con un valor.
- Las cartas se muestran al mismo tiempo.
- Si hay mucha diferencia entre las estimaciones, se explica el motivo.
- Después se vuelve a estimar hasta llegar a un consenso.

Fibonacci

Se suele utilizar la serie:

1 – 2 – 3 – 5 – 8 – 13 – 21...

- Los números crecen cada vez más porque cuanto más grande es una historia, mayor es la incertidumbre.
- Si una User Story tiene demasiados Story Points, probablemente sea demasiado grande para un Sprint y haya que dividirla.
- Si algo tiene valor 0, puede significar que existe desconocimiento y primero hay que investigarlo.

Velocity

- La velocidad mide cuánto trabajo logra completar un equipo durante una iteración.
- Se calcula sumando los Story Points de las User Stories que fueron completadas al 100%.
- No se cuentan las historias parcialmente terminadas.

Ejemplo:

Sprint:

- US1 = 3 SP → terminada
- US2 = 5 SP → terminada
- US3 = 8 SP → no terminada

Velocity = 3 + 5 = 8 SP

- La velocidad sirve para mejorar las estimaciones futuras y también para tener una idea de cuánto puede tardar el proyecto.

- Aproximadamente:

Duración $\approx$ Story Points totales / Velocity

Estimación y empirismo

En Agile se relaciona con:

Ejecutar $\rightarrow$ Inspeccionar $\rightarrow$ Adaptar $\rightarrow$ Feedback

Las estimaciones pueden ir mejorando con la experiencia que vamos obteniendo en cada Sprint.

Gestión de productos

- Acá empezamos a pensar en el producto desde su visión.
- No se trata solamente de "hacer lo que pidió el cliente", sino de entender por qué estamos construyendo el producto y qué valor queremos generar.

¿Por qué creamos productos?

Puede ser para:

- satisfacer clientes,
- generar dinero,
- conseguir usuarios,
- mejorar una situación,
- resolver un problema,
- incluso tener una visión más ambiciosa de cambiar algo en la vida cotidiana.

Problema del desperdicio

- Muchas veces se invierte mucho esfuerzo en funcionalidades que después no se utilizan.
- Por eso hay que intentar minimizar el desperdicio.
- No se trata de construir muchas cosas, sino de construir lo que realmente aporta valor.

User Value Proposition (UVP)

- Propuesta de valor para el usuario.
- Busca entender:
 - qué necesita el usuario,
 - qué problema tiene,
 - qué valor podemos darle.
- A partir de esto se plantea una hipótesis sobre el producto.

MVP – Minimum Viable Product

- Producto Mínimo Viable.
- Se construye lo mínimo necesario para poder validar una hipótesis.
- No significa hacer un producto malo o incompleto.
- La idea es gastar el menor esfuerzo posible para obtener aprendizaje y feedback.

Importante:

MVP ≠ producto terminado.

El objetivo principal es validar si la hipótesis funciona.

Build – Experiment – Learn

Ciclo de aprendizaje:

Construir → Experimentar → Aprender → volver a ajustar

- Construyo algo pequeño.
- Lo pruebo con usuarios.
- Obtengo feedback.
- Aprendo.
- Ajusto el producto y vuelvo a experimentar.

MVF – Minimum Viable Feature

- Es todavía más pequeño que un MVP.
- Se centra en una característica mínima viable.
- La idea es construir rápidamente una feature que aporte valor y comprobar si realmente funciona.
- Si funciona, puedo seguir desarrollando características relacionadas.

MRF – Minimum Release Feature

- Es el conjunto mínimo de características que puedo lanzar como un release.
- Es el incremento más pequeño que aporta un valor nuevo al usuario.

Idea principal de Gestión de Productos

No hay que pensar:

"¿Qué funcionalidades podemos construir?"

Sino más bien:

"¿Qué problema queremos resolver y qué es lo mínimo que podemos construir para comprobar que estamos generando valor?"

El éxito no es simplemente entregar un producto, sino entregar un producto que el usuario realmente utilice y que genere valor.